

Retrieval-Augmented Meta Test-Time Training for Multimodal Reasoning

Shengjie Jin, Zelong Sun, Hengbo Xu, Yuyao Zhang, Dong Jing, Zhiwu Lu*

Gaoling School of Artificial Intelligence

Renmin University of China

Beijing, China

jinshengjie@ruc.edu.cn

luzhiwu@ruc.edu.cn

Abstract

Test-Time Scaling (TTS) methods enhance the reasoning capabilities of Large Vision-Language Models (LVLMs) by allocating additional computation during inference. However, they primarily exploit the model’s internal knowledge, which limits their effectiveness when a query requires information beyond the model’s capacity. While retrieval can introduce such information, existing methods prioritize semantic similarity over reasoning utility and often fail to internalize retrieved content into the model’s usable knowledge, thus limiting its benefit for complex reasoning. To address these limitations, we propose **RAM-TTT (Retrieval-Augmented Meta Test-Time Training)**, a retrieve-train-generate framework that unifies retrieval with meta test-time training. RAM-TTT includes two key components: (1) **LVLM Aligned Retrieval (LAR)**, which re-ranks candidates by both semantic relevance and model-aligned reasoning utility, and (2) **Meta Test-Time Training (Meta TTT)**, which constructs leave-one-out support/query episodes over the retrieved set so that test-time updates generalize across examples instead of overfitting to any single one. Experiments show consistent gains on MathVerse (+6.4%), LogicVista (+5.6%), and We-Math (+8.5%) with Qwen2-VL-7B, together with positive transfer to Phi-3.5-Vision and Pixtral-12B. These findings suggest that RAM-TTT enables LVLMs to actively acquire and internalize new information at test time for stronger multimodal reasoning.

Keywords

Large Vision-Language Model, Multimodal Reasoning, Test-Time Training, Meta-training

1 Introduction

Reasoning is central to Large Vision-Language Models (LVLMs) [5, 16, 32], underpinning tasks such as mathematical problem solving [17, 40] and logical reasoning [35]. Recent advances in Test-Time Scaling (TTS) [20, 28] show that allocating more computation at inference enhances reasoning, for instance by sampling multiple reasoning trajectories [6, 33] or through iterative refinement [18]. However, such approaches often falter when the query requires knowledge beyond the model’s capacity [3], as they recycle the same internal knowledge without introducing new information, leaving the model’s reasoning under-activated [13]. This limitation motivates our goal of equipping test-time scaling with the ability to incorporate external reasoning signals, enabling LVLMs to acquire and internalize new information in such knowledge-gap cases.

A straightforward approach is to incorporate retrieval [3, 15], which can introduce new knowledge and thus partially alleviate this limitation. Yet similarity-based retrieval is often misaligned with reasoning utility: retrieved examples may look relevant but contribute little to solving the query.

Furthermore, even when useful retrieved examples are available, LVLMs often struggle to internalize new information from them [1, 26]. Test-Time Training (TTT) [30] updates model parameters at inference and has shown promise for encoder-based [14, 25, 39] and sequence models [10, 29]. However, direct TTT on a handful of retrieved examples can easily overfit the few-shot supervision and fail to generalize to the query. Thus, an effective test-time paradigm must couple retrieval with robust adaptation, enabling LVLMs to fully exploit retrieved examples without sacrificing generalization.

To address these challenges, we propose **RAM-TTT (Retrieval-Augmented Meta Test-Time Training)**, a multimodal retrieve-train-generate framework that unifies retrieval with meta TTT. RAM-TTT is guided by two principles: (1) retrieval should capture both semantic relevance and alignment with the LVLM’s reasoning; (2) TTT should avoid overfitting on limited supervision while enabling the model to internalize new information at test time.

Building on these principles, we develop two key components. First, **LVLM Aligned Retrieval (LAR)** selects examples not only for semantic relevance but also for their contribution to the LVLM’s reasoning, ensuring that the retrieved examples can enhance reasoning. Second, **Meta Test-Time Training (Meta TTT)** borrows the episodic support/query structure used in meta-learning [27], but instantiates it directly at test time rather than through a separate offline meta-training stage. Concretely, it casts retrieved examples into leave-one-out support/query episodes in the In-Context Learning (ICL) format. This design mitigates overfitting and reduces adaptation difficulty under limited supervision, allowing the LVLM to better learn from retrieved examples. Together, these components integrate retrieval with TTT, enabling LVLMs to both acquire and internalize new information for stronger reasoning under knowledge gaps. Extensive experiments demonstrate clear improvements on MathVerse (+6.4%), LogicVista (+5.6%), and We-Math (+8.5%) with Qwen2-VL-7B. Moreover, we observe positive transfer to other LVLMs such as Phi-3.5-Vision and Pixtral-12B, demonstrating effectiveness beyond a single backbone.

Our main contributions are summarized as follows: (1) We propose RAM-TTT, a unified retrieve-train-generate framework that equips LVLMs to acquire and internalize new information at test time for stronger reasoning. (2) We design LAR, a model-aligned

*Corresponding author.

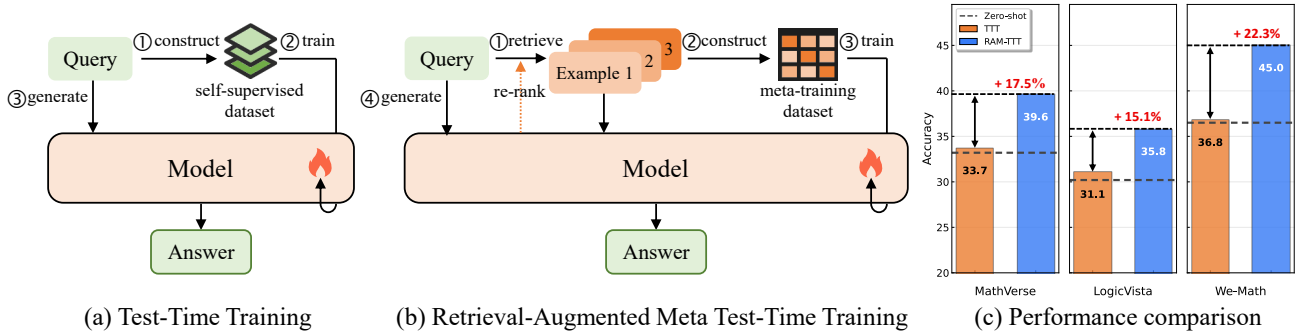


Figure 1: Comparison of TTT and our proposed RAM-TTT. (a) TTT updates the model based on the query, often using self-supervision, without leveraging extra examples in model training. (b) RAM-TTT first retrieves examples according to the query, re-ranks them with the LVLm, and then constructs a meta-training dataset for TTT. After the model is trained, the answer to the query is generated with the retrieved examples as context. (c) RAM-TTT consistently outperforms TTT across benchmarks.

utility re-ranking strategy that selects examples not only by semantic relevance but also by their reasoning utility for the model. (3) We develop a Meta TTT paradigm that casts retrieved examples into leave-one-out support/query episodes in the ICL format, effectively mitigating overfitting and improving generalization to the query.

2 Related Work

Large Vision-Language Models for Multimodal Reasoning. Large Vision-Language Models (LVLms) [5, 16, 32] have made strong progress, yet remain challenged on complex multimodal reasoning, including mathematical problem solving [17, 40] and logical reasoning [35]. Prior work improves reasoning via specialized fine-tuning on reasoning data [12, 24] and chain-of-thought style prompting (including multimodal variants) [34, 41]. Yet they often require retraining or prompt engineering and leave open how to inject query-specific external evidence during inference.

Test-Time Scaling. Recent advances in Test-Time Scaling (TTS) demonstrate that allocating additional computation during inference can substantially improve reasoning performance without retraining [20, 28]. Self-Consistency [33] aggregates multiple reasoning chains, Tree-of-Thoughts [38] performs structured search, and Self-Correction [18] revises initial solutions. However, these methods largely recycle internal knowledge. RAM-TTT instead studies a different point in the design space: multimodal retrieval-augmented adaptation, where external examples are retrieved first and then internalized through test-time updates.

Retrieval-Augmented In-Context Learning. In-Context Learning (ICL) enables models to improve task performance by conditioning on retrieved examples [2, 4, 15]. By incorporating explicit reasoning steps in examples, ICL can boost models’ reasoning abilities [34]. However, two gaps remain in the multimodal setting. First, similarity-based retrieval (e.g., CLIP-based) can favor examples that are visually similar but reasoning-irrelevant. Second, LVLms often struggle to exploit rich multimodal contexts through prompting alone [1, 7, 26]. This leaves open whether parameter-level adaptation after retrieval can provide benefits beyond inference-only multimodal RAG or ICL pipelines.

Test-Time Training. Test-Time Training (TTT) adapts models to specific queries by updating parameters during inference [30, 36]. It has shown effectiveness in improving robustness across vision [30], vision-language [14, 25, 39], and sequence models [10, 29]. Most TTT methods rely on self-supervised signals from the test query, which can be insufficient for multimodal reasoning when the key issue is not only search depth but also access to compatible external evidence. This leaves open how to regularize TTT when supervision comes from a small retrieved support set rather than from the query alone.

Meta-Training. Meta-learning aims to enable models to quickly adapt to new tasks with limited supervision by “learning to learn” [9]. Recent advances extend this paradigm to language models: MetaICL [19, 27] leverages meta-training across diverse tasks to improve few-shot In-Context Learning. While prior work introduces episodic adaptation during pretraining or fine-tuning, our setting is different: we apply a support/query style episodic structure directly at test time over retrieved examples, using it to regularize query-specific adaptation rather than to meta-train on an offline task distribution.

Taken together, these lines of work leave a specific gap: test-time scaling improves search but not external evidence access, retrieval-based ICL provides evidence but often underuses it, and vanilla TTT can adapt but overfits when the support set is tiny. RAM-TTT addresses this gap by tightly integrating model-aligned multimodal retrieval with an episodic test-time training objective, allowing the model to retrieve reasoning-relevant evidence and robustly internalize it via query-specific parameter updates.

3 Methodology

In this work, we propose **Retrieval-Augmented Meta Test-Time Training (RAM-TTT)**, a retrieve-train-generate framework for improving LVLm performance on complex reasoning tasks at test time. As illustrated in Figure 2, RAM-TTT integrates two key components. First, **LVLm Aligned Retrieval (LAR)** retrieves examples that are not only semantically relevant to the query but also compatible with the current LVLm. Second, **Meta Test-Time Training**

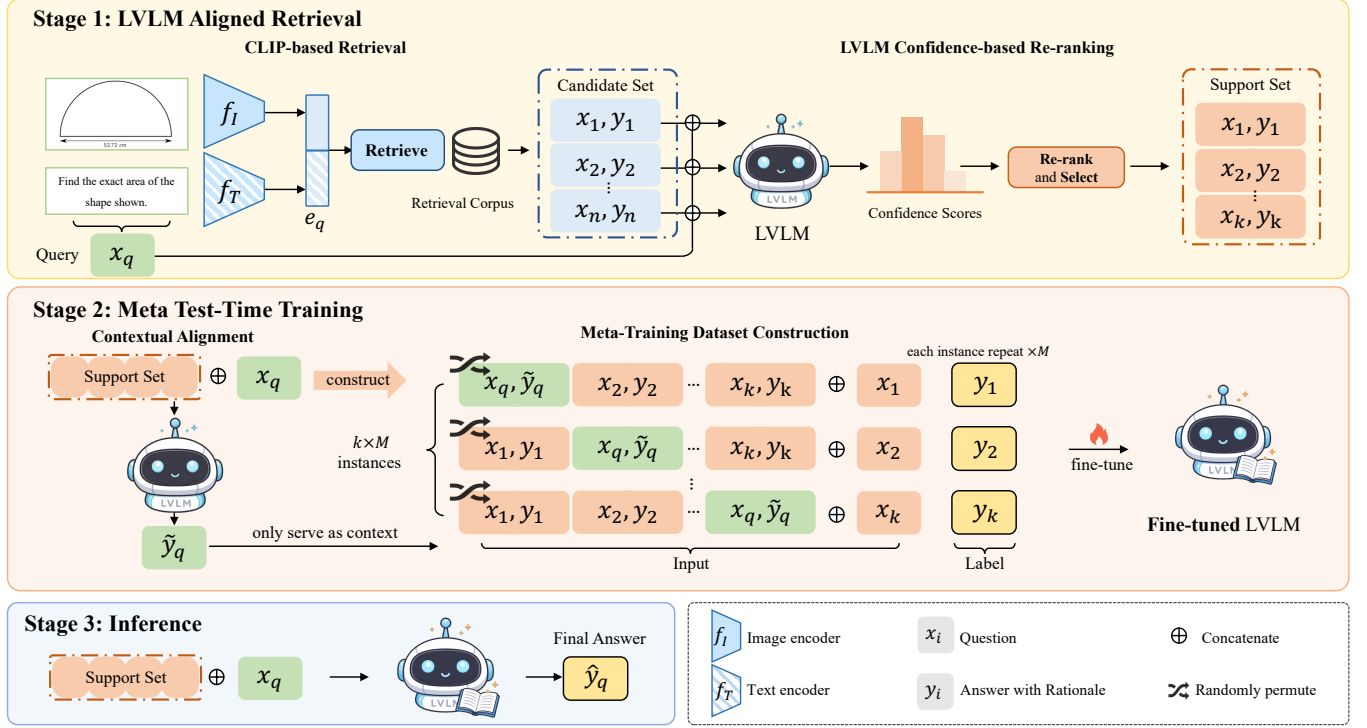


Figure 2: Overview of our RAM-TTT framework. The proposed framework contains two novel components: (1) LVLM Aligned Retrieval (LAR): given a query, retrieve relevant and LVLM-aligned examples from the external corpus; (2) Meta Test-Time Training (Meta TTT): adapt the LVLM to the query using contextual alignment and meta-training on the support set. Once the LVLM is adapted, we use it to generate the final answer for the query.

(Meta TTT) converts the retrieved support set into leave-one-out episodes: in each episode, one retrieved example is held out as the supervision target, while the remaining examples serve as the in-context support for predicting it. This test-time support/query construction encourages updates that transfer across retrieved examples instead of fitting each example in isolation. After adaptation, the updated LVLM is used to answer the original query.

3.1 Problem Setup

Multimodal reasoning tasks, such as mathematical problem solving [17, 31, 40] and logical reasoning [35], typically combine visual and textual information in the query and require sophisticated reasoning to arrive at the correct answer. Formally, let a query be $x_q = (I_q, T_q)$, where I_q is the input image and T_q is the textual question. We have access to a retrieval corpus $C = \{(x_i, y_i)\}_{i=1}^N$ consisting of N examples, where each example is a question-answer pair (x_i, y_i) . Here, $x_i = (I_i, T_i)$ consists of an image I_i , a textual question T_i , and y_i is its corresponding answer with step-by-step rationales. The primary goal is to predict the answer $\hat{y}_q$ for a given query x_q with an LVLM by leveraging relevant examples from the corpus C at test time. Formally, this can be expressed as:

$$\hat{y}_q = \text{LVLM}(\mathbf{I}_{\text{std}}, \mathcal{S}, x_q), \quad (1)$$

where $\mathcal{S}$ is the support set of relevant examples retrieved from C for the query x_q . Here, $\mathbf{I}_{\text{std}}$ refers to the standard ICL instruction prompt,

which guides the model to learn from the provided examples and generate both the reasoning process and the answer for the query. The detailed prompt of $\mathbf{I}_{\text{std}}$ is provided in the appendix.

3.2 LVLM Aligned Retrieval

Given an input query $x_q = (I_q, T_q)$, LAR is proposed to select a small support set $\mathcal{S}$ from the retrieval corpus C according to x_q . Specifically, LAR operates in two sequential stages: CLIP-based candidate retrieval and LVLM confidence-based re-ranking.

CLIP-based Retrieval. In this stage, we first identify a set of semantically relevant candidates. For the query $x_q = (I_q, T_q)$ and each example $x_i = (I_i, T_i)$ in the corpus C , we use CLIP [23] encoders (f_I, f_T) to extract image and text embeddings, and concatenate them to obtain multimodal representations $e_q = [f_I(I_q); f_T(T_q)]$ and $e_i = [f_I(I_i); f_T(T_i)]$. We compute the cosine similarity $\text{Sim}(x_q, x_i) = \cos(e_q, e_i)$. Based on these scores, we select the top- m most similar examples to form a candidate set C_{cand} . We set $m = r \times k$, where k is the final desired support set size and r is a retrieval ratio hyperparameter. After this stage, the examples in the candidate set exhibit high semantic relevance to the query.

LVLM Confidence-based Re-ranking. In this stage, we re-rank candidates in C_{cand} using the LVLM confidence score. For each candidate (x_i, y_i) , we construct an in-context learning prompt $P_i = [\mathbf{I}_{\text{dir}}, x_i, y_i, x_q]$, where $\mathbf{I}_{\text{dir}}$ is a direct-answer instruction (see the appendix), and obtain a tentative answer $\hat{y}_q^{(i)} = \text{LVLM}(P_i)$.

Here the LVLM is asked to output only the final answer, not a step-by-step rationale. Eq. 2 then scores the likelihood of $\hat{y}_q^{(i)}$ under P_i , so the score reflects how well the candidate (x_i, y_i) supports the model’s response to the query.

We compute this score by normalized log-likelihood, equivalently inverse perplexity. Let $L_i = |\hat{y}_q^{(i)}|$ denote the length of the tentative answer. The confidence score is

$$\text{Conf}_q^{(i)} = \exp\left(\frac{1}{L_i} \sum_{t=1}^{L_i} \log p(\hat{y}_t^{(i)} \mid P_i, \hat{y}_{<t}^{(i)})\right). \quad (2)$$

Higher confidence indicates lower average perplexity under P_i , so re-ranking favors candidates that are more compatible with the current model and query. The appendix gives the formal derivation. We finally select the top- k candidates to form the support set $\mathcal{S}$.

3.3 Meta Test-Time Training

After obtaining the support set, Meta TTT adapts the LVLM to the query x_q through query-specific contextual alignment and a leave-one-out episodic objective.

Contextual Alignment Strategy. To incorporate query context into adaptation, we first run the *pre-adaptation* LVLM once with the standard instruction $\mathbf{I}_{\text{std}}$, the support set $\mathcal{S}$, and the query x_q , producing a contextual alignment response $\tilde{y}_q$. We then augment the support set *only for meta-training* as $\mathcal{S}' = \mathcal{S} \cup \{(x_q, \tilde{y}_q)\}$. Because $\tilde{y}_q$ is model-generated rather than annotated, we use it only as an in-context alignment example, never as a supervision target; final inference still uses the original support set $\mathcal{S}$.

Meta-Training Dataset Construction. Our meta-training process uses a leave-one-out strategy over the original support set $\mathcal{S}$. For each ground-truth example $(x_i, y_i) \in \mathcal{S}$, we hold it out as the supervision target, while the context is $\mathcal{S}' \setminus \{(x_i, y_i)\}$, namely the remaining $k - 1$ support examples together with the contextual alignment pair $(x_q, \tilde{y}_q)$. To improve robustness to context order, we generate M random permutations of this context, denoted as $\text{Context}_i^{(l)}$ for $l = 1, \dots, M$. Each prompt is constructed as:

$$P_{\text{meta}}^{(i,l)} = [\mathbf{I}_{\text{std}}, \text{Context}_i^{(l)}, x_i]. \quad (3)$$

The complete meta-training dataset $\mathcal{D}_{\text{meta}}$ is the collection of all such prompt-answer pairs:

$$\mathcal{D}_{\text{meta}} = \left\{ \left(P_{\text{meta}}^{(i,l)}, y_i \right) \mid (x_i, y_i) \in \mathcal{S}, l = 1, \dots, M \right\}. \quad (4)$$

Training Loss. We adapt the LVLM parameters by minimizing the following loss:

$$\mathcal{L}_{\text{TTT}} = - \sum_{(P, y) \in \mathcal{D}_{\text{meta}}} \sum_{t=1}^{|y|} \log p(y_t \mid P, y_{<t}). \quad (5)$$

This yields the adapted parameters θ' .

3.4 Inference

After Meta TTT, the adapted LVLM with parameters θ' is used to generate the final answer. The inference prompt, P_{test} , is constructed using the standard instruction $\mathbf{I}_{\text{std}}$, the original support set $\mathcal{S}$, and the query x_q . The adapted model then predicts the final answer:

$$\hat{y}_q = \text{LVLM}_{\theta'}(P_{\text{test}}). \quad (6)$$

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate RAM-TTT on four widely used multimodal reasoning benchmarks: MathVerse [40], MathVista [17], LogicVista [35], and We-Math [22]. Concretely, MathVerse, MathVista, and We-Math primarily assess visual mathematical reasoning, covering diverse subjects, diagram understanding, and knowledge generalization, while LogicVista focuses on logical reasoning in visual contexts. For MathVerse, we exclude the Vision-Only split and report results on the remaining splits.

Retrieval Corpus Construction. To construct a diverse multimodal retrieval database for RAM-TTT, we process and aggregate MultiMath-300K [21], LLaVA-CoT-100k [37], and Geo170K [11]. For LLaVA-CoT-100k, we retain only the reasoning and conclusion segments. For Geo170K, we keep only entries with chain-of-thought (CoT) annotations and remove duplicates that have the same image. All samples are reformatted into a unified structure: `<think> reasoning process </think> Answer: final answer`, explicitly separating reasoning and answer to help the LVLM use retrieved examples during adaptation. Before evaluation, we further apply deduplication filters to reduce retrieval-time overlap between the retrieval corpus and the benchmark samples. Specifically, we remove entries whose image and text are exactly identical to evaluation samples, and also filter near-duplicates using CLIP ViT-L/14@336px similarity with a threshold above 95%. The final multimodal retrieval corpus used in our experiments contains 403,210 examples after preprocessing and filtering. Further construction details are provided in the appendix.

Backbone and Baselines. We adopt Qwen2-VL-7B [32] as the LVLM backbone and use CLIP ViT-L/14@336px [23] for retrieval. We compare RAM-TTT with the following baselines:

- (1) **Zero-shot:** direct inference with the pretrained LVLM, without adaptation or context.
- (2) **Best of N:** test-time sampling under stochastic decoding of $N=8$ candidate outputs, from which the final answer is selected using the LVLM itself as the scorer [6].
- (3) **Self-Consistency:** test-time sampling under stochastic decoding of $N=8$ outputs followed by consistency-based answer aggregation, where outputs are mapped to final answers and the most frequent one is selected [33].
- (4) **Self-Correction:** test-time iterative refinement of predictions [18].
- (5) **TTT:** parameter-efficient fine-tuning on CLIP-retrieved examples with supervised objectives.
- (6) **ICL:** direct inference with the pretrained LVLM by leveraging k CLIP-retrieved support examples as context [26].

These baselines cover zero-context inference, inference-only retrieval, sampling-based test-time scaling, iterative self-refinement, and retrieval-based test-time adaptation.

Hyperparameters and Implementation Details. We use only 4-shot in-context examples in all main experiments, but explore all few-shot settings in the ablation study. Hyperparameters and implementation details are provided in the appendix. The source code will be released soon.

Table 1: Comparative results on MathVerse and LogicVista. On MathVerse, TD, TL, VI, and VD correspond to Text Dominant, Text Lite, Vision Intensive, and Vision Dominant. On LogicVista, Ded, Ind, Mec, Num, Spa, and ALL correspond to Deductive, Inductive, Mechanical, Numerical, Spatial, and overall accuracy.

Method	MathVerse					LogicVista					
	TD↑	TL↑	VI↑	VD↑	AVG↑	Ded↑	Ind↑	Mec↑	Num↑	Spa↑	ALL↑
Zero-shot	37.3	32.4	31.3	31.6	33.2	51.6	27.1	24.3	26.3	19.2	30.2
Best of N	39.0	34.4	31.2	34.5	34.8	59.1	26.2	31.1	29.5	19.2	33.3
Self-Consistency	36.2	32.7	30.8	32.9	33.2	57.0	29.9	24.3	25.3	26.9	33.1
Self-Correction	42.4	35.3	31.1	32.0	35.2	53.8	22.4	36.5	27.4	26.9	33.1
TTT (4-shots)	36.9	32.9	32.2	32.7	33.7	50.5	28.0	25.7	29.5	19.2	31.1
ICL (4-shots)	42.0	35.5	33.2	33.6	36.1	54.8	24.3	44.6	28.4	15.4	33.3
RAM-TTT (4-shots)	47.5	39.7	36.8	34.4	39.6	55.9	28.0	44.6	30.5	20.5	35.8

Table 2: Comparative results on We-Math and MathVista. On We-Math, IK, IG, CM, and RM correspond to Insufficient Knowledge, Inadequate Generalization, Complete Mastery, and Rote Memorization. On MathVista, TQA, VQA, GPS, MWP, FQA, and ALL correspond to Textbook Question Answering, Visual Question Answering, Geometry Problem Solving, Math Word Problem, Figure Question Answering, and overall accuracy.

Method	We-Math					MathVista					
	IK↓	IG↓	CM↑	RM↓	Overall↑	TQA↑	VQA↑	GPS↑	MWP↑	FQA↑	ALL↑
Zero-shot	56.8	5.5	33.7	10.6	36.5	60.8	55.3	40.9	65.1	65.1	57.6
Best of N	53.0	5.5	35.4	14.5	38.2	62.0	53.1	50.5	64.0	60.2	57.9
Self-Consistency	51.4	7.4	37.7	8.8	41.4	58.9	53.6	53.4	63.4	63.6	58.9
Self-Correction	48.8	12.2	34.9	10.7	41.0	57.6	42.5	45.7	60.2	58.4	53.1
TTT (4-shots)	57.0	5.3	34.1	9.6	36.8	61.4	54.7	42.3	65.1	66.5	58.3
ICL (4-shots)	50.5	10.1	36.0	8.7	41.1	60.8	51.4	45.2	60.2	66.5	57.3
RAM-TTT (4-shots)	44.6	11.1	39.4	11.2	45.0	63.9	53.1	52.4	66.1	63.9	60.0

4.2 Main Results

As summarized in Tables 1 and 2, **RAM-TTT** remains competitive against zero-shot inference, sampling-based test-time scaling, iterative self-refinement, and retrieval-based baselines. We view these results as evidence that the combination of LAR and Meta TTT is effective on the evaluated settings. The experimental analysis for each benchmark is detailed below.

On **MathVerse**, RAM-TTT surpasses Zero-shot and TTT by 6.4% and 5.9% in average accuracy, respectively. The largest gains appear on the Text-dominant (+10.2%) and Text-lite (+7.3%) splits, which suggests that retrieval-augmented adaptation is particularly useful when the query can benefit from related textual reasoning traces. The gains on the vision-intensive splits are smaller but still positive, indicating that the method remains helpful when visual evidence plays a larger role.

On **LogicVista**, RAM-TTT improves overall accuracy over Zero-shot and ICL by 5.6% and 2.5%, respectively. The gains are concentrated in Deductive and Numerical reasoning, while some categories change only marginally. This pattern suggests that Meta TTT helps on several logical structures, but the benefit is not uniform across all LogicVista subskills.

On **We-Math**, RAM-TTT improves overall accuracy by 8.5% over Zero-shot and by 3.9% over ICL. The reduction in Insufficient

Knowledge (IK) errors and the increase in Complete Mastery (CM) suggest that retrieval-augmented adaptation is especially helpful when the base model lacks the needed reasoning knowledge. However, the method does not dominate every diagnostic axis: the Inadequate Generalization (IG) score is worse than TTT, and the Rote Memorization (RM) score is worse than ICL. This pattern indicates that RAM-TTT is better at recovering missing knowledge than at uniformly preventing broader generalization errors or retrieval-induced shortcut behavior.

On **MathVista**, Self-Correction shows a marked drop in overall performance, which may reflect the brittleness of repeated self-refinement without external grounding on visually grounded problems. Best of N and Self-Consistency are more competitive on this benchmark, especially on some reasoning-heavy subsets, indicating that extra inference-time compute alone can recover part of the gap. ICL also underperforms Zero-shot on several splits, particularly TQA and MWP, where limited relevance between test questions and the retrieval corpus exposes the weakness of similarity-based retrieval. RAM-TTT improves the overall score by 2.4% over Zero-shot and yields a large gain on GPS, but the gains are again not uniform: the VQA split remains below Zero-shot and the FQA split remains below TTT/ICL. These regressions suggest that retrieval

Table 3: Ablation study for the main components of RAM-TTT on MathVerse and LogicVista.

Method	MathVerse					LogicVista					
	TD↑	TL↑	VI↑	VD↑	AVG↑	Ded↑	Ind↑	Mec ↑	Num↑	Spa↑	ALL↑
TTT	36.9	32.9	32.2	32.7	33.7	50.5	28.0	25.7	29.5	19.2	31.1
+ Meta TTT	44.4	35.8	33.8	32.4	36.6	54.8	23.4	44.6	33.7	16.7	34.5
+ Conf. Re-ranking	45.7	38.5	34.5	33.2	38.0	54.8	29.0	44.6	30.5	17.9	35.3
+ Contextual Align	47.5	39.7	36.8	34.4	39.6	55.9	28.0	44.6	30.5	20.5	35.8

Table 4: Ablation studies on CLIP models and similarity computation strategies.

(a) CLIP Models						(b) Similarity Computation					
CLIP Models	TD↑	TL↑	VI↑	VD↑	AVG↑	Similarity Comput.	TD↑	TL↑	VI↑	VD↑	AVG↑
ViT-B/32	44.9	36.9	33.0	31.6	36.6	Image Only	41.5	37.3	34.1	32.9	36.5
ViT-B/16	45.2	37.2	34.8	31.7	37.2	Text Only	43.4	36.4	36.0	33.2	37.3
ViT-L/14	44.7	38.6	36.7	32.5	38.1	Mean Similarity	44.7	40.9	37.1	34.0	39.1
ViT-L/14@336px	47.5	39.7	36.8	34.4	39.6	Concat	47.5	39.7	36.8	34.4	39.6

Table 5: Ablation study of examples permutation times M on MathVerse and LogicVista.

M	MathVerse					LogicVista					
	TD↑	TL↑	VI↑	VD↑	AVG↑	Ded↑	Ind↑	Mec ↑	Num↑	Spa↑	ALL↑
1	44.9	39.3	35.0	33.0	38.1	52.7	23.4	44.6	30.5	24.4	34.7
2	47.5	39.7	36.8	34.4	39.6	55.9	28.0	44.6	30.5	20.5	35.8
3	46.3	40.2	36.0	33.2	39.0	52.7	28.0	45.9	30.5	21.8	35.6
4	46.2	40.7	38.1	32.9	39.5	50.5	27.1	44.6	30.5	19.2	34.2

and adaptation are less reliable when the answer depends on fine-grained local visual cues or figure-specific conventions that are weakly matched by retrieved exemplars. We therefore attribute the overall improvement to better retrieval-aligned adaptation on several reasoning-heavy subsets, while acknowledging that some MathVista settings remain retrieval-mismatched.

Collectively, the above main results demonstrate that RAM-TTT’s integration of LAR and Meta TTT facilitates robust and generalizable improvements for complex multimodal reasoning tasks, establishing strong performance for test-time training.

4.3 Ablation Study

Component Analysis. In Table 3, we conduct an incremental ablation study, starting from the basic TTT and progressively adding in turn each core component of RAM-TTT: Meta TTT, confidence-based re-ranking, and contextual alignment. Our findings are as follows: (1) + **Meta TTT**. Adding the Meta TTT module leads to a clear improvement over the basic TTT. This pattern is consistent with the leave-one-out support/query construction, which makes adaptation less sensitive to individual retrieved examples than standard TTT, which directly fine-tunes on the same small support set it uses as supervision. The more direct evidence for improved stability is reported later in Section 5. (2) + **Confidence Re-ranking**. Further incorporating confidence-based re-ranking brings additional

gains, especially on MathVerse (from 36.6% to 38.0%). This highlights the benefit of selecting examples that better align with the LVLm, thereby further enhancing the model’s reasoning behavior. (3) + **Contextual Alignment**. Finally, incorporating the contextual alignment strategy yields the best overall result among all settings in this table. The gain is clear on MathVerse (38.0% to 39.6%) and modest on LogicVista (35.3% to 35.8%), indicating that contextual alignment is helpful.

Effects of CLIP Model and Similarity Computation Strategy. In Table 4 (a), we investigate the impact of different CLIP models on MathVerse. We observe that larger and higher-resolution models yield consistent improvements: ViT-L/14@336px achieves the highest average accuracy (39.6%), outperforming ViT-B/32 (36.6%) and ViT-B/16 (37.2%). Stronger CLIP encoders produce more discriminative joint image-text embeddings, leading to more accurate retrieval candidates that better match the query’s semantic and visual structure, which in turn strengthens downstream adaptation. In Table 4 (b), we further conduct experiments on different similarity computation strategies in the CLIP retrieval stage. We find that only using the image or text features for retrieval leads to lower accuracy (“Image Only” and “Text Only”), demonstrating that multimodal reasoning requires both modalities to be jointly considered. Compared to separately averaging similarities of the two modalities (“Mean Similarity”), concatenating the modal features (“Concat”) yields better performance because it allows the

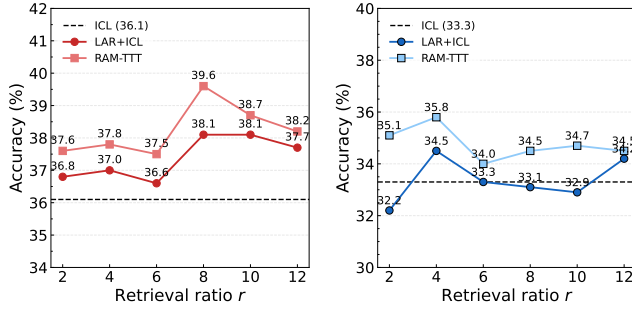


Figure 3: Effect of r on model performance on MathVerse (left) and LogicVista (right).

retrieval to capture cross-modal interactions that are essential for diagram-intensive mathematical reasoning.

Effects of Permutation Times M . Table 5 presents the impact of the number of permutation times M when constructing meta-training samples. We observe that: (1) increasing M from 1 to 2 leads to a clear improvement in overall accuracy on both MathVerse and LogicVista, indicating that moderate permutation helps the model better use the support set by exposing it to diverse in-context example orders; (2) further increasing M to 3 or 4 does not yield additional gains and can slightly decrease performance, suggesting that excessive permutations may introduce redundancy without providing additional meaningful diversity. Overall, setting $M = 2$ is sufficient to achieve strong results.

Effects of Retrieval Ratio r . We further analyze the impact of the retrieval ratio r on model performance, as shown in Figure 3. On MathVerse, RAM-TTT achieves its highest accuracy at $r = 8$ (39.6%), while on LogicVista, the best result is at $r = 4$ (35.8%). Increasing r expands the candidate pool and improves the chance of retrieving high-utility examples, but excessively large r introduces semantically similar yet reasoning-irrelevant examples, adding noise that weakens the meta-updates. Comparing with the CLIP-based Retrieval + ICL baseline (dashed line), RAM-TTT consistently outperforms ICL across the tested r values in our experiments, demonstrating that Meta TTT remains relatively stable even when retrieval contains mild noise. In contrast, LAR+ICL without test-time training remains more sensitive to retrieval noise, particularly on LogicVista.

Generalization to Other Backbones. To test whether RAM-TTT transfers beyond a single LVLM, we extend our evaluation to two additional backbones: Phi-3.5-Vision and Pixtral-12B. As shown in Figure 4, we report representative MathVerse (TD) and MathVista (GPS) splits for these additional backbones. RAM-TTT consistently improves over both zero-shot and CLIP-based ICL baselines on these tested settings. Specifically, it raises Phi-3.5-Vision’s accuracy from 31.7% (Zero-shot) and 36.5% (CLIP Retrieval + ICL) to 38.1%, and improves Pixtral-12B from 45.9% and 48.9% to 50.3%. Similar improvements are also observed on MathVista (GPS). These results indicate positive transfer across diverse LVLM backbones.

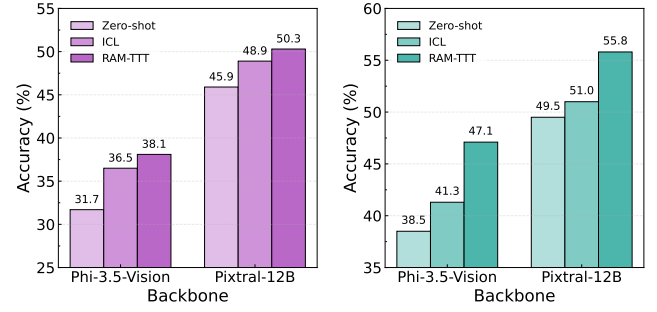


Figure 4: RAM-TTT with different backbones on MathVerse (TD, left) and MathVista (GPS, right).

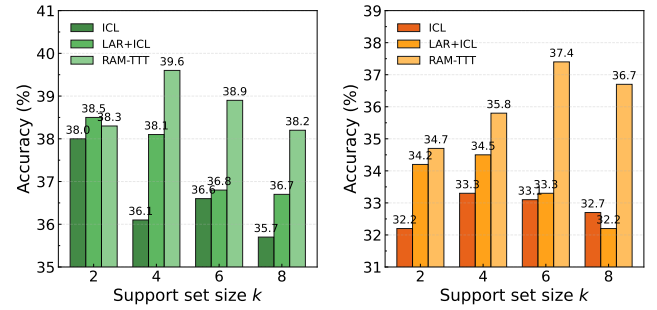


Figure 5: Effect of support set size k on MathVerse (left) and LogicVista (right).

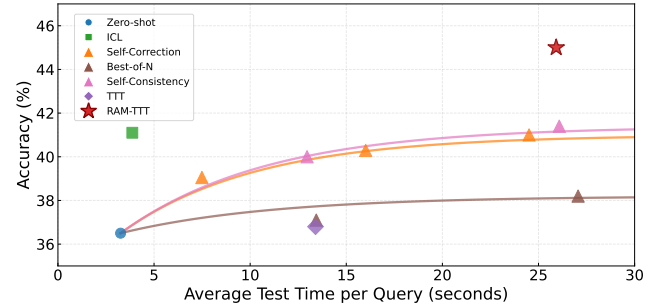


Figure 6: Accuracy-time trade-off across methods.

5 Analysis and Discussion

Beyond reporting benchmark gains, we further analyze *how* RAM-TTT works by examining five aspects: (1) its ability to leverage retrieved samples, (2) its efficiency in converting additional computation into reasoning improvements, (3) its robustness to overfitting as the number of adaptation steps increases, (4) its robustness to prompt design, and (5) the effect of confidence-based example selection. These empirical findings complement our main results, while theoretical justification is deferred to the appendix.

Sample Utilization Analysis. Figure 5 shows the performance of RAM-TTT and two baselines with varying support set size k . This reflects how efficiently different methods can leverage retrieved

examples. We observe two key phenomena: (1) *Monotonic gains under moderate k* . When k increases from 2 to 6, RAM-TTT exhibits consistent improvement, whereas baselines either plateau or degrade. This highlights RAM-TTT’s ability to transform additional examples into meaningful reasoning signals rather than mere noise. (2) *Robustness under larger k* . When k increases further to 8, all methods face accuracy drops due to noise from less relevant examples. However, RAM-TTT only shows a mild decrease, while baselines suffer steeper declines. Together, these findings indicate that RAM-TTT makes more effective use of additional external examples while remaining more tolerant to retrieval noise than the compared baselines. Similar patterns on LogicVista (Figure 5, right) suggest that this behavior is not unique to MathVerse.

Accuracy-Time Trade-off Analysis. Figure 6 presents the trade-off between accuracy and computational time on one NVIDIA A800 GPU. Best-of- N , Self-Consistency, and Self-Correction all benefit from additional test-time compute, but their gains saturate in the high-compute regime. On the We-Math dataset, RAM-TTT remains the highest-accuracy method among the approaches plotted at comparable per-query latency, reaching 45.0% at 25.92 seconds per query, compared with 41.4% for Self-Consistency ($N=8$), 41.0% for Self-Correction ($n=6$), and 38.2% for Best-of- N ($N=8$). The appendix further reports a detailed end-to-end latency breakdown for RAM-TTT, with Meta TTT as the dominant cost and re-ranking/final inference contributing smaller but still non-negligible overheads. These results suggest that, among the methods included in Figure 6, retrieval-augmented adaptation converts additional test-time compute into larger accuracy gains than inference-only scaling alone. Even so, its latency still places RAM-TTT in offline batch evaluation, high-value assessment, and other compute-tolerant reasoning scenarios rather than interactive or latency-critical deployment. We therefore frame RAM-TTT as a practical retrieval-augmented adaptation method for multimodal reasoning, not as a universal real-time solution.

Adaptation Stability and Overfitting Analysis. To evaluate robustness, we plot accuracy as a function of adaptation steps for TTT, TTT with weight decay, and RAM-TTT on MathVista (GPS) (Figure 7). Standard TTT improves performance only slightly in the first few steps and then degrades rapidly, showing how easily direct fine-tuning can overfit tiny support sets. Adding weight decay delays the drop only marginally and fails to ensure long-term stability. RAM-TTT, in contrast, improves in the early steps and remains comparatively stable across a much wider step range. This empirical pattern is consistent with the leave-one-out episodic objective reducing sensitivity to any single retrieved example.

Prompt Robustness Analysis. To verify the prompt robustness of RAM-TTT, we evaluate three prompt types on Qwen2-VL-7B: the original prompt, a simplified prompt, and a dedicated thinking prompt that additionally encourages explicit step-by-step reasoning, as detailed in the appendix. The bar plots in Figure 8 demonstrate that RAM-TTT remains highly stable across these diverse prompt variations. All prompts achieve comparable performance, yielding around 50–53% accuracy on MathVista GPS and 35–38% on LogicVista, with the thinking prompt attaining the overall best results. These findings indicate that RAM-TTT does not critically depend

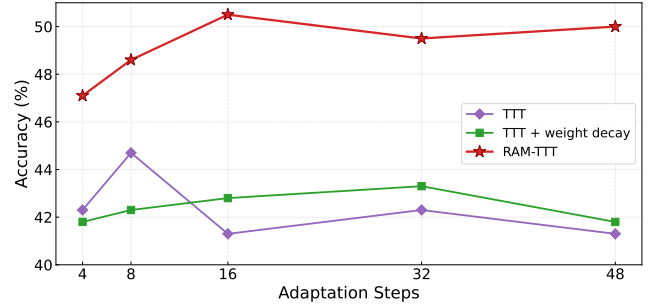


Figure 7: Adaptation stability and overfitting analysis under increasing adaptation steps on MathVista (GPS).

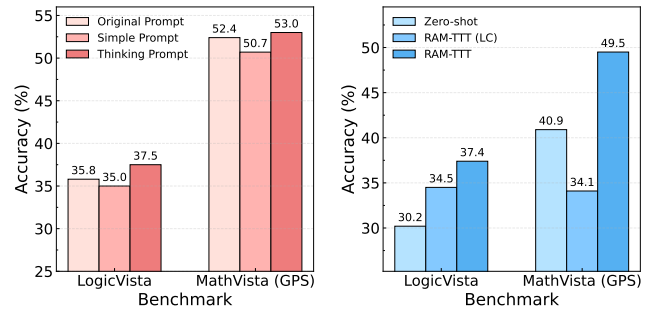


Figure 8: Prompt robustness analysis (left) and the effect of confidence-based example selection (right).

on meticulously crafted prompts and remains reasonably stable under prompt variants once retrieved examples and meta-adaptation are effectively incorporated.

The Effect of Confidence-based Example Selection. LAR ranks candidate examples using the LVLM confidence score introduced in Section 3.2 and selects the top- k items as the support set. To directly assess this design, we compare RAM-TTT with high-confidence selection against a variant that deliberately uses the lowest-confidence candidates (both with $k=6$). As shown in Figure 8, low-confidence selection substantially hurts performance on MathVista (GPS), dropping to 34.1%, which is even worse than the zero-shot baseline at 40.9%, whereas high-confidence selection reaches 49.5%. On LogicVista, low-confidence achieves 34.5% compared to 37.4% for high-confidence and 30.2% for zero-shot. These results support our view that inverse perplexity is a useful proxy for reasoning utility.

6 Conclusion

This paper introduces RAM-TTT, a retrieve-train-generate framework designed to enhance LVLMs for reasoning via test-time training. By integrating LVLM-Aligned Retrieval (LAR), which combines CLIP-based retrieval with LVLM Confidence-based Re-ranking, RAM-TTT selects examples that are both query-relevant and aligned with the LVLM’s reasoning, effectively addressing the misalignment between retrieval relevance and reasoning utility that plagues existing retrieval-augmented methods. Furthermore,

RAM-TTT employs Meta Test-Time Training (Meta TTT), featuring a Contextual Alignment Strategy and leave-one-out episodic updates in the ICL format, to adapt the LVLm with these examples while mitigating overfitting. Extensive experiments demonstrate consistent gains and model-agnostic improvements across LVLm backbones on complex multimodal reasoning.

References

- [1] Folco Bertini Baldassini, Mustafa Shukor, Matthieu Cord, Laure Soulier, and Benjamin Piwowarski. 2024. What makes multimodal in-context learning work?. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1539–1550.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.
- [3] Davide Caffagni, Federico Cocchi, Nicholas Moratelli, Sara Sarto, Marcella Cornia, Lorenzo Baraldi, and Rita Cucchiara. 2024. Wiki-llava: Hierarchical retrieval-augmented generation for multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 1818–1826.
- [4] Wenhui Chen, Hexiang Hu, Xi Chen, Pat Verga, and William W Cohen. 2022. Murag: Multimodal retrieval-augmented generator for open question answering over images and text. *arXiv preprint arXiv:2210.02928* (2022).
- [5] Zhe Chen, Jiannan Wu, Wenhui Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. 2024. Internvl: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 24185–24198.
- [6] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168* (2021).
- [7] Sivan Doherty, Shaked Perer, M Jehanzeb Mirza, Wei Lin, Amit Alfassy, Assaf Arbelle, Shimon Ullman, and Leonid Karlinsky. 2024. Towards multimodal in-context learning for vision & language models. *arXiv preprint arXiv:2403.12736* (2024).
- [8] Haodong Duan, Junming Yang, Yuxuan Qiao, Xinyu Fang, Lin Chen, Yuan Liu, Xiaoyi Dong, Yuhang Zang, Pan Zhang, Jiaqi Wang, et al. 2024. Vlmevalkit: An open-source toolkit for evaluating large multi-modality models. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 11198–11201.
- [9] Chelsea Finn, Pieter Abbeel, and Sergey Levine. 2017. Model-agnostic meta-learning for fast adaptation of deep networks. In *International conference on machine learning*. PMLR, 1126–1135.
- [10] Yossi Gandelsman, Yu Sun, Xinlei Chen, and Alexei Efros. 2022. Test-time training with masked autoencoders. *Advances in Neural Information Processing Systems* 35 (2022), 29374–29385.
- [11] Jiahui Gao, Renjie Pi, Jipeng Zhang, Jiacheng Ye, Wanjuan Zhong, Yufei Wang, Lanqing Hong, Jianhua Han, Hang Xu, Zhenguo Li, et al. 2023. G-llava: Solving geometric problem with multi-modal large language model. *arXiv preprint arXiv:2312.11370* (2023).
- [12] Jarvis Guo, Tuney Zheng, Yuelin Bai, Bo Li, Yubo Wang, King Zhu, Yizhi Li, Graham Neubig, Wenhui Chen, and Xiang Yue. 2024. Mammoth-vl: Eliciting multimodal reasoning with instruction tuning at scale. *arXiv preprint arXiv:2412.05237* (2024).
- [13] Peng Hu, Changjiang Gao, Ruiqi Gao, Jiajun Chen, and Shujian Huang. 2024. Large Language Models are Limited in Out-of-Context Knowledge Reasoning. *arXiv preprint arXiv:2406.07393* (2024).
- [14] Yuto Kojima, Jiarui Xu, Xueyan Zou, and Xiaolong Wang. 2025. LoRA-TTT: Low-Rank Test-Time Training for Vision-Language Models. *arXiv preprint arXiv:2502.02069* (2025).
- [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [16] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Zhiwei Liu, et al. 2024. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326* (2024).
- [17] Pan Lu, Hritik Bansal, Tony Xia, Jiacheng Liu, Chunyuan Li, Hannaneh Hajishirzi, Hao Cheng, Kai-Wei Chang, Michel Galley, and Jianfeng Gao. 2023. Mathvista: Evaluating mathematical reasoning of foundation models in visual contexts. *arXiv preprint arXiv:2310.02253* (2023).
- [18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems* 36 (2023), 46534–46594.
- [19] Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2021. Metaicl: Learning to learn in context. *arXiv preprint arXiv:2110.15943* (2021).
- [20] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393* (2025).
- [21] Shuai Peng, Di Fu, Liangcai Gao, Xiuqin Zhong, Hongguang Fu, and Zhi Tang. 2024. Multimath: Bridging visual and mathematical reasoning for large language models. *arXiv preprint arXiv:2409.00147* (2024).
- [22] Runqi Qiao, Qiuna Tan, Guanting Dong, Minhui Wu, Chong Sun, Xiaoshuai Song, Zhuoma GongQue, Shanglin Lei, Zhe Wei, Miaoquan Zhang, et al. 2024. We-math: Does your large multimodal model achieve human-like mathematical reasoning? *arXiv preprint arXiv:2407.01284* (2024).
- [23] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*. PMLR, 8748–8763.
- [24] Wenhao Shi, Zhiqiang Hu, Yi Bin, Junhua Liu, Yang Yang, See-Kiong Ng, Lidong Bing, and Roy Ka-Wei Lee. 2024. Math-llava: Bootstrapping mathematical reasoning for multimodal large language models. *arXiv preprint arXiv:2406.17294* (2024).
- [25] Manli Shu, Wei Nie, De-An Huang, Zhiding Yu, Tom Goldstein, Anima Anandkumar, and Chaowei Xiao. 2022. Test-time prompt tuning for zero-shot generalization in vision-language models. *Advances in Neural Information Processing Systems* 35 (2022), 14274–14289.
- [26] Mustafa Shukor, Alexandre Rame, Corentin Dancette, and Matthieu Cord. 2023. Beyond task performance: Evaluating and reducing the flaws of large multimodal models with in-context learning. *arXiv preprint arXiv:2310.00647* (2023).
- [27] Sanchit Sinha, Yuguang Yue, Victor Soto, Mayank Kulkarni, Jianhua Liu, and Aiding Zhang. 2024. Maml-en-llm: Model agnostic meta-training of llms for improved in-context learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2711–2720.
- [28] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).
- [29] Yu Sun, Xinhao Li, Karan Dalal, Jiarui Xu, Arjun Vikram, Genghan Zhang, Yann Dubois, Xinlei Chen, Xiaolong Wang, Sanmi Koyejo, et al. 2024. Learning to (learn at test time): Rnns with expressive hidden states. *arXiv preprint arXiv:2407.04620* (2024).
- [30] Yu Sun, Xiaolong Wang, Zhuang Liu, John Miller, Alexei Efros, and Moritz Hardt. 2020. Test-time training with self-supervision for generalization under distribution shifts. In *International conference on machine learning*. PMLR, 9229–9248.
- [31] Ke Wang, Juntao Pan, Weikang Shi, Zimu Lu, Mingjie Zhan, and Hongsheng Li. 2024. Measuring multimodal mathematical reasoning with math-vision dataset. *arXiv preprint arXiv:2402.14804* (2024).
- [32] Peng Wang, Shuai Bai, Sinan Tan, Shijie Jiang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. 2024. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191* (2024).
- [33] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2022. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171* (2022).
- [34] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [35] Yijia Xiao, Edward Sun, Tianyu Liu, and Wei Wang. 2024. Logicvista: Multimodal llm logical reasoning benchmark in visual contexts. *arXiv preprint arXiv:2407.04973* (2024).
- [36] Zehao Xiao and Cees GM Snoek. 2024. Beyond model adaptation at test time: A survey. *arXiv preprint arXiv:2411.03687* (2024).
- [37] Guowei Xu, Peng Jin, Li Hao, Yibing Song, Lichao Sun, and Li Yuan. 2024. Llava-o1: Let vision language models reason step-by-step. *arXiv preprint arXiv:2411.10440* (2024).
- [38] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems* 36 (2023), 11809–11822.
- [39] Hee Suk Yoon, Eunseop Yoon, Joshua Tian Jin Tee, Mark Hasegawa-Johnson, Yingzhen Li, and Chang D Yoo. 2024. C-tp: Calibrated test-time prompt tuning for vision-language models via text feature dispersion. *arXiv preprint arXiv:2403.14119* (2024).
- [40] Renrui Zhang, Dongzhi Jiang, Yichi Zhang, Haokun Lin, Ziyu Guo, Pengshuo Qiu, Aojun Zhou, Pan Lu, Kai-Wei Chang, Yu Qiao, et al. 2024. Mathverse: Does your multi-modal llm truly see the diagrams in visual math problems?. In *European Conference on Computer Vision*. Springer, 169–186.

- [41] Zhuosheng Zhang, Aston Zhang, Mu Li, Hai Zhao, George Karypis, and Alex Smola. 2023. Multimodal chain-of-thought reasoning in language models. *arXiv preprint arXiv:2302.00923* (2023).
- [42] Yuze Zhao, Jintao Huang, Jinghan Hu, Xingjun Wang, Yunlin Mao, Daoze Zhang, Zeyinzi Jiang, Zhikai Wu, Baole Ai, Ang Wang, Wenmeng Zhou, and Yingda Chen. 2024. SWIFT: A Scalable lightWeight Infrastructure for Fine-Tuning. *arXiv preprint arXiv:2408.05517* (2024).

A Implementation Details

Hardware and Software. Experiments were conducted on 4 NVIDIA A800 GPUs with 80GB memory each, using Ubuntu 20.04, Python 3.10, PyTorch 2.5.1+cu121, and ms-swift 3.1.0.dev0 [42]¹.

Training Hyperparameters. Unless otherwise specified, all experiments shared common training configurations. For model components, LLM parameters were unfrozen for training, while the ViT and Aligner components remained frozen.

The core training hyperparameters of our method are selected as follows:

- **Max sequence length:** 4096
- **LoRA rank:** 8
- **LoRA alpha:** 8
- **Batch size:** 1
- **Optimizer:** AdamW
- **Learning rate scheduler:** Cosine
- **Learning rate:** 5×10^{-5} or 2×10^{-5}
- **Epochs:** 1 or 2
- **Mixed precision:** bf16
- **Random seed:** 42

The specific learning rates and number of training epochs are detailed as follows:

- **MathVerse:** learning rate 5×10^{-5} , 2 epochs
- **LogicVista:** learning rate 5×10^{-5} , 1 epoch
- **MathVista:** learning rate 2×10^{-5} , 2 epochs
- **We-Math:** learning rate 2×10^{-5} , 2 epochs

Evaluation. We evaluate all models using VLMEvalKit [8]², an open-source evaluation toolkit for large vision-language models.

B Prompt Details

Zero-shot. For zero-shot inference, the model was provided with the image followed directly by the question, without any additional instructional text. Our prompt differs slightly from the standard VLMEvalKit setup, as we removed instructions such as “Please directly answer the question and provide the correct option letter, e.g., A, B, C, D.”

Prompt-symbol mapping. In the main paper, I_{std} denotes the standard ICL instruction used during contextual alignment, Meta TTT, and final inference, while I_{dir} denotes the direct-answer instruction used only for confidence-based re-ranking. Figures 9 and 10 provide the concrete templates corresponding to these two symbols.

In-Context Learning (ICL). The prompt format for ICL, which was consistent for both training (during Meta TTT) and testing, is detailed in Figure 9. In the notation of Section 3, this figure instantiates I_{std} .

¹<https://github.com/modelscope/ms-swift>

²<https://github.com/open-compass/VLMEvalKit>

You will be provided questions and answers similar to the ones below. Please study these examples carefully to understand the knowledge, concepts, and reasoning required to solve such problems. These examples serve as reference material containing important information. After the examples, I will give you a new question. Apply the knowledge you've learned from these examples to answer the new question accurately.

Example 1: <image> Question: {question 1} Answer: {answer 1}
Example 2: <image> Question: {question 2} Answer: {answer 2}
Example 3: <image> Question: {question 3} Answer: {answer 3}
Example 4: <image> Question: {question 4} Answer: {answer 4}

<image> Question: {test question}

Figure 9: Prompt format used for In-Context Learning (ICL) during training and testing.

Confidence Score Calculation. When calculating confidence scores for the Model-Informed Re-ranking phase, the model was given the candidate support example and the test question. An additional instruction was provided to guide the model to output only the final answer. The specific prompt for this can be seen in Figure 10; this prompt corresponds to I_{dir} in Section 3. The tentative answer $\hat{y}_q^{(i)}$ is the single direct-answer sequence generated under this prompt with the same single-pass inference configuration used in normal answer generation, and the confidence score is computed from the token-level log-likelihood of that generated sequence.

You will be provided questions and answers similar to the ones below. Please study these examples carefully to understand the knowledge, concepts, and reasoning required to solve such problems. These examples serve as reference material containing important information. After the examples, I will give you a new question. Apply the knowledge you've learned from these examples to answer the new question accurately.

Example 1: <image> Question: {question 1} Answer: {answer 1}

<image> Question: {test question}

Please provide only your answer directly (e.g., a letter, a number, or a short phrase). Do not include any additional text or formatting.

Figure 10: Prompt format used for instructing the model to output only the final answer during confidence score calculation.

Self-Correction. For the Self-Correction baseline, a specific instruction was designed to prompt the model to review its initial answer and make corrections if necessary. This instructional prompt is illustrated in Figure 11.

C Retrieval Corpus Construction

To build a large and diverse retrieval database for multimodal reasoning, we aggregate and process data from three major sources: MultiMath-300K [21], LLaVA-CoT-100k [37], and Geo170K [11]. Below, we detail the filtering, deduplication, and standardization procedures applied to each dataset.

Deduplication to reduce retrieval-time overlap risk. To reduce retrieval-time overlap between the retrieval corpus and the benchmark samples used in evaluation, we further filter the corpus against the benchmark samples. We remove entries whose image and text

```

<image> Question: {test question}

My initial answer: {response}

Let me analyze my initial answer:
1. Is my answer accurate and complete?
2. Did I miss any important details in the image?
3. Are there any logical errors or inconsistencies in my reasoning?
4. Is my answer directly addressing what was asked?

I'll analyze step by step and then provide a refined answer:

```

Figure 11: Instructional prompt used for the Self-Correction baseline.

```

You will be provided questions and answers similar to the ones below.

Example 1: <image> Question: {question 1} Answer: {answer 1}
Example 2: <image> Question: {question 2} Answer: {answer 2}
Example 3: <image> Question: {question 3} Answer: {answer 3}
Example 4: <image> Question: {question 4} Answer: {answer 4}

<image> Question: {test question}

```

Figure 12: A simplified template that only keeps the minimal ICL structure.

```

You will be provided questions and answers similar to the ones below. Let's think step by
step to answer the last question accurately.

Example 1: <image> Question: {question 1} Answer: {answer 1}
Example 2: <image> Question: {question 2} Answer: {answer 2}
Example 3: <image> Question: {question 3} Answer: {answer 3}
Example 4: <image> Question: {question 4} Answer: {answer 4}

<image> Question: {test question}

```

Figure 13: A “thinking” template that adds a standard step-by-step instruction.

are exactly identical to evaluation samples. We also compute similarity with CLIP ViT-L/14@336px and filter near-duplicates whose similarity exceeds 95%. These rules materially lower the chance of retrieving leaked benchmark questions while still preserving the diversity of the retrieval corpus, but they do not formally preclude every hidden overlap pattern. They also target retrieval-corpus overlap only, rather than any separate pretraining-data overlap.

MultiMath-300K. We use the full MultiMath-300K dataset, which contains a wide variety of multimodal math problems with detailed solutions. No additional filtering is applied, as the dataset already provides high-quality, well-structured question-answer pairs with accompanying images.

LLaVA-CoT-100k. The LLaVA-CoT-100k dataset provides rich multimodal question-answer pairs, where each response is annotated with four pairs of special tags: <SUMMARY>...</SUMMARY>, <CAPTION>...</CAPTION>, <REASONING>...</REASONING>, and <CONCLUSION>...</CONCLUSION>. These tags serve distinct purposes:

- <SUMMARY>: Summarizes the overall approach or strategy for solving the problem.

- <CAPTION>: Describes relevant content or details from the associated image.
- <REASONING>: Contains the step-by-step reasoning or chain-of-thought process leading to the answer.
- <CONCLUSION>: Presents the final answer or conclusion derived from the reasoning.

For our retrieval corpus, we extract only the <REASONING> and <CONCLUSION> segments from each response. This choice is motivated by our focus on providing the LVLM with high-quality, explicit reasoning traces and clear final answers, which are most beneficial for in-context learning and test-time training. The <SUMMARY> and <CAPTION> tags, while informative, are omitted to maintain a concise and consistent format across all retrieval examples.

Geo170K. Geo170K contains a mix of multimodal geometry problems, some with and some without explicit reasoning. We filter the dataset to retain only those entries annotated with chain-of-thought (CoT) rationales. Additionally, to avoid redundancy and potential bias, we remove duplicate samples that share the same image, keeping only one instance per unique image.

Data Standardization and Formatting. After filtering, all samples from the three sources are reformatted into a unified structure to facilitate consistent in-context demonstration for the LVLM. Each example is organized as follows: <think> reasoning process </think>

Answer: final answer

Corpus Statistics. The final retrieval corpus consists of 298,677 examples from MultiMath-300K, 98,571 from LLaVA-CoT-100k, and 5,962 from Geo170K, totaling 403,210 examples.

D Benchmark Datasets

We comprehensively evaluate our method on four diverse multimodal reasoning benchmarks, covering mathematical, logical, and spatial reasoning across varied contexts and task formats.

MathVerse. [40] A visual math benchmark emphasizing diagram understanding and reasoning under varying text-vision balances. We use the “testmini” split (3,940 samples), focusing on four subsets (TD, TL, VI, VD) with progressively less text and more visual reliance. The VO subset is excluded due to its image-only format.

LogicVista. [35] A logical reasoning benchmark with 448 questions spanning five tasks: Inductive, Deductive, Numerical, Spatial, and Mechanical reasoning. It supports both open-ended and multiple-choice evaluation, probing complementary reasoning skills in visual contexts.

We-Math. [22] A benchmark targeting principles of visual mathematical reasoning, with 1,740 samples. It introduces a four-dimensional metric—Insufficient Knowledge, Inadequate Generalization, Complete Mastery, and Rote Memorization—enabling fine-grained analysis beyond accuracy. We report results under the loose setting.

MathVista. [17] A large-scale benchmark (1,000 “testmini” samples) covering algebra, geometry, statistics, logic, and scientific reasoning. It integrates 28 multimodal datasets and three new ones (IQTest, FunctionQA, PaperQA), spanning natural images, diagrams, charts, and synthetic scenes, thus ensuring broad coverage of visual-mathematical reasoning tasks.

E Theoretical Analysis

This section formalizes the local inductive bias of RAM-TTT through three objects: the query-conditioned score used by LAR, the held-out episodic risk minimized by Meta TTT, and the prompt perturbation induced by contextual alignment. The analysis is explanatory rather than universal. It clarifies why RAM-TTT prefers reasoning-compatible candidates and held-out cross-example transfer, but it does not claim correctness guarantees, universal robustness, or a formal proof that contamination risk is eliminated.

For any prompt-response pair (P, y) , define the sequence negative log-likelihood and its length-normalized version as

$$\ell_\theta(P, y) = -\sum_{t=1}^{|y|} \log p_\theta(y_t | P, y_{<t}), \quad \bar{\ell}_\theta(P, y) = \frac{1}{|y|} \ell_\theta(P, y). \quad (7)$$

E.1 LVLM Confidence-based Re-ranking

For each candidate (x_i, y_i) , LAR builds a prompt $P_i = [\mathbf{I}_{\text{dir}}, x_i, y_i, x_q]$ and obtains a tentative answer $\hat{y}_q^{(i)} = (\hat{y}_1^{(i)}, \dots, \hat{y}_{T_i}^{(i)})$. We then define

$$\mathcal{L}_q^{(i)} = \bar{\ell}_\theta(P_i, \hat{y}_q^{(i)}), \quad \text{PPL}_q^{(i)} = \exp(\mathcal{L}_q^{(i)}), \quad \text{Conf}_q^{(i)} = \exp(-\mathcal{L}_q^{(i)}). \quad (8)$$

It is convenient to work with the log-confidence score

$$s_i = \log \text{Conf}_q^{(i)} = -\mathcal{L}_q^{(i)}. \quad (9)$$

Proposition 1 (Ranking equivalence). For any two candidates i and j ,

$$\text{Conf}_q^{(i)} \geq \text{Conf}_q^{(j)} \iff \mathcal{L}_q^{(i)} \leq \mathcal{L}_q^{(j)} \iff \text{PPL}_q^{(i)} \leq \text{PPL}_q^{(j)}.$$

Therefore, selecting the top- k candidates by confidence is equivalent to selecting the k candidates with the smallest length-normalized sequence loss under the query-conditioned prompt.

Proof. The mapping $z \mapsto \exp(z)$ is strictly increasing, so ordering by $\text{PPL}_q^{(i)} = \exp(\mathcal{L}_q^{(i)})$ is identical to ordering by $\mathcal{L}_q^{(i)}$. Likewise, $\text{Conf}_q^{(i)} = \exp(-\mathcal{L}_q^{(i)})$ is a strictly decreasing transform of $\mathcal{L}_q^{(i)}$, so larger confidence is equivalent to smaller loss. $\square$

Remark (Standard ranking stability). The top- k set is stable under bounded score perturbations whenever the log-confidence margin $s_{(k)} - s_{(k+1)}$ is sufficiently larger than the perturbation magnitude. This is a standard property of score-based ranking and clarifies that small numerical variations do not necessarily change the selected support set.

The key takeaway of this subsection is therefore modest: LAR ranks candidates by query-conditioned normalized sequence loss. This supports the claim that confidence-based re-ranking favors candidates that are more compatible with the model under the current prompt, but it does not imply that high-confidence candidates are always correct or universally useful.

E.2 Episodic Meta TTT and Held-out Risk

Let $\mathcal{S} = \{(x_i, y_i)\}_{i=1}^k$ be the retrieved support set, and let $\mathcal{S}' = \mathcal{S} \cup \{(x_q, \tilde{y}_q)\}$ be the augmented set after contextual alignment. For each target $(x_i, y_i) \in \mathcal{S}$ and permutation index $l \in \{1, \dots, M\}$, Meta

TTT constructs the prompt $P_{\text{meta}}^{(i,l)}$ in Eq. (4). Define the episode loss

$$\ell_{i,l}^{\text{meta}}(\theta) = \ell_\theta(P_{\text{meta}}^{(i,l)}, y_i). \quad (10)$$

The corresponding empirical held-out episodic risk is

$$R_{\text{held}}(\theta) = \frac{1}{kM} \sum_{i=1}^k \sum_{l=1}^M \ell_{i,l}^{\text{meta}}(\theta). \quad (11)$$

Observation (Held-out episodic risk). By construction,

$$\mathcal{L}_{\text{TTT}}(\theta) = kM \cdot R_{\text{held}}(\theta),$$

so Meta TTT directly minimizes the empirical average loss of predicting each retrieved example from a context that excludes that example itself. This is the narrow sense in which the procedure is “meta”: each retrieved pair acts as a held-out meta-query while the remaining examples form the temporary support context.

Shortcut gap. For comparison, let $P_{\text{self}}^{(i,l)}$ denote an otherwise matched prompt whose context also contains the target pair (x_i, y_i) , and define

$$\ell_{i,l}^{\text{self}}(\theta) = \ell_\theta(P_{\text{self}}^{(i,l)}, y_i), \quad \delta_{i,l}(\theta) = \ell_{i,l}^{\text{meta}}(\theta) - \ell_{i,l}^{\text{self}}(\theta).$$

When self-inclusion creates a trivial shortcut, such as making it easier to copy the target pattern from its own context, $\delta_{i,l}(\theta)$ becomes positive and the self-including objective is optimistically biased relative to the held-out one. The leave-one-out construction removes this most direct shortcut, although it does not rule out all forms of overfitting to shared templates across the retrieved support set.

E.3 Relation to Meta-learning Intuition

The held-out episodic construction above is closely related to the support/query decomposition used in meta-learning [9, 27] and in meta-level analyses of in-context adaptation [19]. The connection in RAM-TTT is structural rather than algorithmic: like meta-learning, the method separates temporary support context from held-out query targets, so the update is encouraged to transfer across examples instead of reconstructing the same support item it conditions on.

At the same time, RAM-TTT does *not* solve an explicit bilevel meta-objective or compute second-order meta-gradients. Its implementation is the single-level aggregated loss in Eq. (5). We therefore use “Meta TTT” to describe the episodic support/query inductive bias of the objective, not to claim that the method is a full MAML-style optimizer.

E.4 Contextual Alignment and Prompt Mismatch

Contextual alignment augments each episode with the pseudo pair $(x_q, \tilde{y}_q)$, which is used only as context. To isolate its role, let $C_{i,l}$ denote the leave-one-out episode context without the pseudo pair, and define the corresponding base and aligned prompts as

$$P_{i,l}^{\text{base}} = [\mathbf{I}_{\text{std}}, C_{i,l}, x_i], \quad P_{i,l}^{\text{align}}(z) = [\mathbf{I}_{\text{std}}, C_{i,l}, (x_q, z), x_i]. \quad (12)$$

The difference between $P_{i,l}^{\text{base}}$ and $P_{i,l}^{\text{align}}(\tilde{y}_q)$ is purely a change in prompt context: the aligned version makes the episode more similar to the final inference prompt by exposing the model to the current

Table 6: Per-query latency of each component in RAM-TTT.

Component	Avg. Latency (s)
CLIP-based retrieval	0.43
LVLm re-ranking	4.31
Contextual alignment	0.59
Meta TTT	16.17
Final inference	4.42
Total (RAM-TTT)	25.92

query during adaptation. When $\tilde{y}_q$ is a reasonable query-side hint, this can reduce train-test prompt mismatch. When $\tilde{y}_q$ is poor, the same mechanism can steer the episode toward an unhelpful reasoning trajectory.

This interpretation is intentionally modest. Because $(x_q, \tilde{y}_q)$ is never used as a supervised target, contextual alignment does not inject pseudo-label noise through direct label fitting; its influence enters through context conditioning. This is consistent with the empirical pattern in Table 3: contextual alignment helps on average, but the gains are small and not uniformly positive on every split.

E.5 Boundary Conditions and Limitations

What the analysis supports. The analysis above supports three limited conclusions. First, LAR ranks candidates by query-conditioned normalized log-likelihood, which makes the confidence score an explicit compatibility measure under the current prompt. Second, Meta TTT minimizes an empirical held-out episodic risk, so its support/query split is materially different from directly fine-tuning on self-including prompts. Third, contextual alignment changes prompt structure in a way that can reduce train-test mismatch when the pseudo-label is useful, while still exposing the method to context-induced failure modes when the pseudo-label is poor.

What the analysis does not claim. The analysis does not prove that high-confidence candidates are correct, that Meta TTT universally generalizes across all reasoning benchmarks, or that contextual alignment always helps. It also does not establish that our deduplication fully eliminates hidden overlap, nor does it imply that RAM-TTT is low-cost in practice. These issues remain empirical and are discussed in Sections 4, 5, and F.

F Computational Overhead Analysis

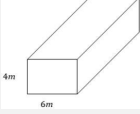
We provide the per-query computational cost of each component in RAM-TTT. All measurements were conducted on a single NVIDIA A800 GPU using Qwen2-VL-7B as the backbone.

Meta TTT is the dominant cost because it performs query-specific parameter updates. The remaining stages are materially cheaper but still non-negligible, especially LVLm re-ranking and final inference. We therefore view RAM-TTT as most appropriate for compute-tolerant reasoning settings where additional latency is acceptable in exchange for improved accuracy.

G Case Study

Qualitative examples help reveal how RAM-TTT changes the reasoning trajectory, not just the final answer.

Question:
Find the surface area of the rectangular prism with length equal to 12 m.



Zero-shot(×):
The surface area of a rectangular prism is 2 times the product of the length and the width plus 2 times the product of the length and the height plus 2 times the product of the width and the height. In this case, the length is 12 m and the width is 4 m. We don't know the height, but we can find it by using the fact that the volume of a rectangular prism is equal to the length times the width times the height. In this case, the volume is equal to 12 m times 4 m times the height, or 48 m^3 . We can then solve for the height by dividing both sides of the equation by 12 m times 4 m, which gives us the height of 2 m. Now we can use the height to find the surface area. The surface area is 2 times the product of the length and the width plus 2 times the product of the length and the height plus 2 times the product of the width and the height, or 2 times 12 m times 4 m plus 2 times 12 m times 2 m plus 2 times 4 m times 2 m, which is equal to 160 m^2 .

ICL(×):
<think>Step 1 (Surface area formula): According to the surface area formula of a rectangular prism, the surface area is the sum of the areas of each face.
Step 2 (First two faces area): Each 4×4 square has an area of $4 \times 4 = 16$, and there are two of these squares, so we have 2×16 .
Step 3 (Bottom and top face areas): The length is 10, the width is 4, so each bottom and top face has an area of $10 \times 4 = 40$, and the two bottom and top faces have a total area of $2 \times 40 = 80$.
Step 4 (Side face areas): The length is 12, the width is 4, so each side face has an area of $12 \times 4 = 48$, and the two side faces have a total area of $2 \times 48 = 96$.
Step 5 (Total surface area): The total surface area is the sum of the areas of each face, so the total surface area is $2 \times 16 + 80 + 96 = 208$.
</think>Answer: \boxed{208}

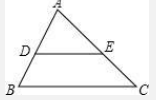
RAM-TTT(✓):
<think>Step 1 (Surface area formula): The formula for the surface area of a rectangular prism is $2(\text{Length} \times \text{Width} + \text{Length} \times \text{Height} + \text{Width} \times \text{Height})$.
Step 2 (Substitute the values): $2(12 \text{ m} \times 4 \text{ m} + 12 \text{ m} \times 6 \text{ m} + 4 \text{ m} \times 6 \text{ m})$.
Step 3 (Calculate individual areas): $2(48 \text{ m}^2 + 72 \text{ m}^2 + 24 \text{ m}^2)$.
Step 4 (Calculate surface area): $2(144 \text{ m}^2) = 288 \text{ m}^2$.</think>
Answer: \boxed{288 \text{ m}^2}

Figure 14: Case study 1: Zero-shot, ICL, and RAM-TTT.

Figure 14 shows a representative progression in reasoning quality. Zero-shot identifies the relevant formula but misreads the inputs, which leads to a hallucinated intermediate computation and an incorrect answer. ICL adopts the step-by-step format but still mixes information from the retrieved examples into the current query. RAM-TTT preserves the useful reasoning structure while grounding each step in the query-specific evidence, producing both the correct intermediate computation and the final answer.

Figure 15 shows a similar pattern on a different problem type. Zero-shot gives an unsupported answer, and ICL partially organizes the derivation but still introduces mathematical errors. RAM-TTT instead maintains a coherent chain of reasoning from the visual evidence to the final solution, illustrating that the method adapts the reasoning behavior to the current query rather than simply copying the surface form of retrieved demonstrations.

Question:
Question: As shown in the figure, it is known that angle A = 80.0, angle B = 60.0, DE parallel BC, then the size of angle CED is ()
Choices:
A: 40°
B: 60°
C: 120°
D: 140°



Zero-shot(×):
A

ICL(×):
<think>Step 1 (Triangle Exterior Angle): Calculate the exterior angle of $\angle A$: $\angle A + \angle ACB = 180^\circ$.
Step 2 (Substitute the known angles): $80^\circ + \angle ACB = 180^\circ$.
Step 3 (Calculate the difference): $\angle ACB = 180^\circ - 80^\circ = 100^\circ$.
Step 4 (Parallel lines property): Since $DE \parallel BC$, based on the property of alternate interior angles being equal, $\angle CED = \angle ACB = 100^\circ$.
</think>Answer: \boxed{100^\circ}

RAM-TTT(✓):
<think>Step 1 (Sum of angles theorem): In $\triangle ABC$, $\angle A + \angle B + \angle C = 180^\circ$.
Step 2 (Substitute known angles): $\angle A = 80^\circ$, $\angle B = 60^\circ$.
Step 3 (Calculate $\angle C$): $\angle C = 180^\circ - 80^\circ - 60^\circ = 40^\circ$.
Step 4 (Alternate interior angles with parallel lines): Since $DE \parallel BC$, $\angle CED$ and $\angle C$ are alternate interior angles and therefore equal.
Step 5 (Determine $\angle CED$ value): $\angle CED = 180^\circ - \angle C = 180^\circ - 40^\circ = 140^\circ$.
</think>Answer: \boxed{140^\circ}

Figure 15: Case study 2: Zero-shot, ICL, and RAM-TTT.